TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 2: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block (wrapped in `try/except NotImplementedError`) with tests. Implement the missing functions so the tests pass and the script prints `"All tests passed!"`. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions (`random_array`, `is_sorted`, `GROWTH_FUNCTIONS`, `generate_instance`, and the Week-1-style stable-matching helpers) live in `starter_code.py` at the week root.

The 17 problems are grouped into four themes:

- **Asymptotic notation and recurrences** (Problems 1–6): ordering functions by growth rate, witness checkers for $O/\Omega/\Theta$, and directly simulating the three canonical recurrences from the lecture notes.
- **Searching** (Problems 7–9): binary search (iterative and recursive), first/last occurrence, and search in a rotated sorted array.
- **Selection and data structures** (Problems 10–15): max/min with a comparison counter, median via sorting vs. quickselect, binary heaps, an indexed priority queue, heap vs. sorted-list priority queues, and amortized analysis of a dynamic array.
- **Algorithm analysis case studies** (Problems 16–17): an efficient array/queue-based re-implementation of Gale–Shapley, and correctness-via-induction examples (loop invariants and binary exponentiation).

1 Asymptotic Notation and Recurrences

Problem 1 – Ordering Functions by Growth Rate

Implement `order_by_growth(functions, n)` that takes a dict mapping names to callables of n (see `GROWTH_FUNCTIONS` in `starter_code.py`: constant, logarithmic, sqrt, linear, linearithmic, quadratic, cubic, exponential) and returns the list of names sorted by *increasing* value at the given n (slowest- to fastest-growing, at that particular n).

Then implement `dominates(f, g, n_values)` that returns `True` iff, for every n in `n_values` (an increasing list of positive integers), $f(n) \leq g(n)$. This is a simple empirical proxy for "f is $O(g)$" (it does not prove it, but a violation immediately disproves it).

Expected behavior. At $n = 512$, the order should match the textbook hierarchy: constant, logarithmic, sqrt, linear, linearithmic, quadratic, cubic, exponential. Also, `dominates(logarithmic, linear, [1,2,4,...,128])` and `dominates(linear, quadratic, ...)` should be `True`, while `dominates(quadratic, linear, ...)` should be `False`.

Problem 2 – Counting Operations in Nested Loops

Implement three "instrumented" functions that count the number of times their innermost operation runs, as a function of n:

- `count_single_loop(n)`: a single loop `for i in range(n)`, one operation per iteration. Returns exactly n.
- `count_nested_loop(n)`: a loop `for i in range(n)` containing an inner loop `for j in range(n)`, one operation per inner iteration. Returns exactly n^2.
- `count_triangular_loop(n)`: a loop `for i in range(n)` containing an inner loop `for j in range(i)` (note: `range(i)`, not `range(n)`), one operation per inner iteration. Returns exactly $0 + 1 + \cdots + (n-1) = \frac{n(n-1)}{2}$.

Then implement `tightest_bound(counts, n_values)` that, given a list of operation counts measured at the corresponding `n_values`, returns the name of the tightest matching growth class from {`"constant"`, `"linear"`, `"quadratic"`}, by checking which of `n_values[i]**0`, `n_values[i]**1`, `n_values[i]**2` the counts are closest to being proportional to (the ratio `counts[i] / n_values[i]**k` should be roughly constant across all i, for the smallest k that works, within a relative tolerance of 0.15).

Expected behavior. `tightest_bound` applied to the outputs of `count_single_loop`, `count_nested_loop`, and `count_triangular_loop` (over several n) returns `"linear"`, `"quadratic"`, and `"quadratic"` respectively.

Problem 3 – Witness Checkers for O, Ω, and Θ

Implement `is_big_o(f, g, c, n0, n_values)` that returns `True` iff $f(n) \leq c \cdot g(n)$ holds for *every* n in `n_values` with $n \geq n0$ (values $n < n0$ are ignored – the definition only requires the bound to hold eventually).

Similarly implement `is_big_omega(f, g, c, n0, n_values)` ($f(n) \geq c\, g(n)$ for $n \geq n0$), and `is_theta(f, g, c_lo, c_hi, n0, n_values)` which returns `True` iff $c\_lo \cdot g(n) \leq f(n) \leq c\_hi \cdot g(n)$ for all n in `n_values` with $n \geq n0$.

These are *witness checkers*: given proposed constants $(c, n0)$ or $(c\_lo, c\_hi, n0)$, they verify the inequality holds on a finite sample of n – a necessary (but not sufficient) condition for a full proof, useful for sanity-checking candidate constants before writing a proof.

Expected behavior. For $f(n) = 5n^2 + 3n + 2$ and $g(n) = n^2$: `is_big_o(f, g, c=6, n0=5, ...)` is `True` (matching Theory Question A2); `is_big_omega(f, g, c=5, n0=0, ...)` is `True` (matching A3); and `is_theta(f, g, c_lo=5, c_hi=6, n0=5, ...)` is `True`.

Problem 4 – Recurrence $T(n) = T(n-1) + O(1)$: Linear Recursion

Consider the recursive function `countdown_ops(n)` that simulates the recurrence $T(n) = T(n-1)+1$ (with $T(0) = 1$):

```
def countdown_ops(n):
    if n <= 0:
        return 1
    return 1 + countdown_ops(n - 1)
```

The closed form is $T(n) = n + 1 = \Theta(n)$.

Implement `countdown_ops(n)` exactly as above (recursively – do not just return `n+1` directly; the point is to call this on small n and see that the *number of recursive calls* matches the closed form).

Then implement `verify_linear_recurrence(n_values)` that, for each n in `n_values`, computes `countdown_ops(n)` and checks it equals $n + 1$. Return `True` iff this holds for every n in `n_values`.

Expected behavior. `verify_linear_recurrence([0,1,5,10,50])` returns `True`.

Problem 5 – Recurrence $T(n) = T(n/2) + O(1)$: Logarithmic Recursion

Consider the recursive function `halving_ops(n)` that simulates the recurrence $T(n) = T(\lfloor n/2 \rfloor)+1$ for $n \geq 1$, with $T(0) = 1$:

```
def halving_ops(n):
    if n <= 0:
        return 1
    return 1 + halving_ops(n // 2)
```

The closed form is $T(n) = \Theta(\log n)$; precisely, $T(n) = \lfloor \log_2 n \rfloor + 2$ for $n \geq 1$ ($T(0) = 1$).

Implement `halving_ops(n)` recursively as above.

Then implement `verify_log_recurrence(n_values)` that, for each n in `n_values` ($n \geq 1$), computes `halving_ops(n)` and checks it equals $\lfloor \log_2 n \rfloor + 2$. Return `True` iff this holds for every n in `n_values`.

Expected behavior. `verify_log_recurrence([1,2,4,8,16,100,1000])` returns `True`.

Problem 6 – Recurrence $T(n) = 2T(n/2) + O(n)$: Linearithmic Recursion

Consider the recursive function `divide_combine_ops(n)` that simulates the recurrence $T(n) = 2T(n/2) + n$ for $n > 1$, with $T(1) = 1$ (and $T(0) = 0$):

```
def divide_combine_ops(n):
    if n <= 1:
        return n
    return 2 * divide_combine_ops(n // 2) + n
```

This is the same recurrence that governs mergesort's running time, and its solution is $T(n) = \Theta(n \log n)$.

Implement `divide_combine_ops(n)` recursively as above. (For n that is not a power of 2, `n // 2` truncates – that's fine, just follow the recurrence literally.)

Then implement `verify_linearithmic_growth(powers_of_two)` where `powers_of_two` is a list of exponents k (so $n = 2^k$). For each k, compute $T(n) =$ `divide_combine_ops`(2^k). When n is an exact power of two, the closed form is $T(n) = n \cdot (k + 1) = n \log_2 n + n$. Check that `divide_combine_ops(2**k) == 2**k * (k+1)` for every k in `powers_of_two`. Return `True` iff this holds for all of them.

Expected behavior. `verify_linearithmic_growth([0,1,2,3,4,5,6,7,8,9,10])` returns `True`.

2 Searching

Problem 7 – Binary Search (Iterative and Recursive)

Implement two versions of binary search over a sorted list `arr`:

- `binary_search_iterative(arr, target)`: returns the index of `target` in `arr`, or -1 if not present. Use a `while` loop.
- `binary_search_recursive(arr, target)`: same behaviour, implemented recursively (a helper with `lo`/`hi` bounds is fine).

Also implement `recursion_depth(n)`, returning the number of recursive calls `binary_search_recursive` would make in the *worst case* on a sorted array of length n, i.e. $\lceil \log_2(n+1) \rceil$ – the number of halvings until the search range becomes empty.

Expected behavior. On `arr = [1,3,5,7,9,11,13,15]`, both search functions return the correct index for present values (e.g. `target=7` $\rightarrow$ index 3) and -1 for absent values. `recursion_depth(15)` returns 4.

Problem 8 – First and Last Occurrence (Binary Search Variant)

Given a sorted array `arr` that may contain duplicate values, implement:

- `find_first(arr, target)`: returns the smallest index i such that `arr[i] == target`, or -1 if `target` is not present.
- `find_last(arr, target)`: returns the largest such index, or -1.
- `count_occurrences(arr, target)`: returns the number of times `target` appears in `arr`, computed as `find_last(arr, target) - find_first(arr, target) + 1` if present, else 0.

Each of `find_first` and `find_last` must run in $O(\log n)$ time: do a binary search that, upon finding `target`, continues searching the left half (for `find_first`) or right half (for `find_last`) instead of returning immediately, to find the boundary.

Expected behavior. On `arr = [1,2,2,2,3,4,5]`, `find_first(arr,2) == 1`, `find_last(arr,2) == 3`, `count_occurrences(arr,2) == 3`, and `count_occurrences(arr,9) == 0`.

Problem 9 – Search in a Rotated Sorted Array

An array `arr` of distinct integers, originally sorted in increasing order, has been "rotated" at some unknown pivot, e.g. `[4,5,6,7,0,1,2]` (originally `[0,1,2,4,5,6,7]`, rotated by 4).

Implement `search_rotated(arr, target)` that returns the index of `target` in `arr`, or -1 if not present, in $O(\log n)$ time.

Hint. At any point during binary search, at least one of the two halves `[lo..mid]` or `[mid..hi]` is "normally sorted" (its first element $\leq$ its last element). Determine which half is sorted, then check whether `target` could lie in that half's range to decide which half to recurse/iterate into.

Expected behavior. `search_rotated([4,5,6,7,0,1,2], 0) == 4`, `search_rotated([4,5,6,7,0,1,2], 3) == -1`.

3 Selection and Data Structures

Problem 10 – Finding Max and Min with a Comparison Counter

Implement `find_max_min(arr)` that returns `(max_val, min_val, comparisons)` where `comparisons` is the total number of element-vs-element comparisons performed, using the *naive* approach: scan once to find the max ($n-1$ comparisons), then scan again to find the min ($n-1$ comparisons), for a total of $2(n-1)$.

Then implement `find_max_min_paired(arr)` using the classic "divide into pairs" trick: process elements two at a time. For each pair (a, b), first compare a vs. b (1 comparison) to determine which is the candidate-max and which is the candidate-min of the pair, then compare the candidate-max against the running maximum (1 comparison) and the candidate-min against the running minimum (1 comparison) – 3 comparisons per pair, instead of 4. Handle an odd-length array by initializing the running max/min from the first element (0 extra comparisons) before processing the rest in pairs.

Expected behavior. For n elements, `find_max_min` uses $2(n-1)$ comparisons and `find_max_min_paired` uses roughly $\frac{3n}{2}$ – fewer for $n > 2$. Both return the same `(max_val, min_val)`.

Problem 11 – Median via Sorting vs. Quickselect

Implement `median_via_sort(arr)` that returns the median of `arr` (defined as `sorted(arr)[(len(arr)-1)//2]` for both even and odd lengths) by calling Python's built-in `sorted()`.

Then implement `quickselect(arr, k)`, which returns the k-th smallest element (0-indexed) of `arr` using the Quickselect algorithm:

1. If `len(arr) == 1`, return `arr[0]`.
2. Choose a pivot (use `arr[0]` for determinism).
3. Partition `arr` into three lists: elements `< pivot`, `== pivot`, and `> pivot`.
4. If `k < len(less)`, recurse into `less` with the same `k`. Elif `k < len(less) + len(equal)`, return `pivot`. Else recurse into `greater` with `k - len(less) - len(equal)`.

Then implement `median_via_quickselect(arr)` that calls `quickselect(arr, (len(arr)-1)//2)`.

Finally, implement `count_comparisons_sort(arr)` returning the number of comparisons `sorted()` would roughly need in the worst case for an array of length n, i.e. `ceil(n * log2(n))` if $n > 1$ else 0 – a standard upper-bound estimate for comparison sorts, for *discussion*, not measured directly.

Expected behavior. `median_via_sort(arr) == median_via_quickselect(arr)` for a variety of arrays, including arrays with duplicates.

Problem 12 – Binary Min-Heap from Scratch

Implement an array-based binary `MinHeap` class with:

- `push(x)`: insert `x`, then "sift up" to restore the heap property.
- `pop()`: remove and return the minimum element, moving the last element to the root and "sifting down" to restore the heap property. Raise `IndexError` if empty.
- `peek()`: return (without removing) the minimum element. Raise `IndexError` if empty.

- `__len__`: number of elements.

The heap is stored in `self.data` (a Python list), where for index i the children are at indices $2i+1$ and $2i+2$, and the parent is at $(i-1)//2$.

Then implement `heapsort(arr)` that returns a new sorted list by pushing all elements of `arr` onto a `MinHeap` and popping them off in order. This gives an $O(n \log n)$ sort.

Expected behavior. `heapsort([5,3,1,4,2]) == [1,2,3,4,5]`; popping from a `MinHeap` always returns elements in non-decreasing order.

Problem 13 – Indexed Priority Queue with `decrease_key`

In algorithms like Dijkstra's shortest path (covered in a later week), we need a priority queue where items can have their priority *decreased* after insertion, and we need to look up an item's current position efficiently.

Implement `IndexedMinPQ`, a binary min-heap of `(key, priority)` pairs that supports:

- `insert(key, priority)`: insert a new key with the given priority. Assumes `key` is not already present.
- `decrease_key(key, new_priority)`: lower the priority of an existing `key` to `new_priority` (assumes `new_priority <=` current priority), then restore the heap property by sifting up.
- `extract_min()`: remove and return `(key, priority)` with the smallest priority, restoring the heap property by sifting down. Raise `IndexError` if empty.
- `__len__`: number of elements.
- `__contains__(key)`: `True` iff `key` is currently in the priority queue.

Implementation hints. Maintain `self.heap`: a list of `[priority, key]` pairs (heap-ordered by priority). Maintain `self.position`: a dict mapping `key -> index in self.heap`, updated on every swap, so `decrease_key` can find the key in $O(1)$ and then sift up in $O(\log n)$.

Expected behavior. After inserting several `(key, priority)` pairs, `extract_min` returns them in increasing order of (possibly decreased) priority, and `decrease_key` followed by `extract_min` reflects the updated priority.

Problem 14 – Heap-Based PQ vs. Naive Sorted-List PQ

A priority queue can be implemented in (at least) two ways:

1. **Heap-based** (Problem 12's `MinHeap`): `push` and `pop` are both $O(\log n)$.
2. **Naive sorted list**: maintain a Python list that is kept sorted at all times.
   - `push(x)`: find the insertion point and insert – count `len(self.data)` "shift operations" *before* insertion (worst case, inserting at the front).
   - `pop()`: remove and return `self.data.pop(0)` – count `len(self.data) - 1` "shift operations" *before* removal (the number of remaining elements that must shift left).

Implement `SortedListPQ` with `push`, `pop`, `peek`, `__len__`, and an `self.ops` counter that accumulates the "shift operations" described above.

Then implement `compare_pq_costs(n, seed)`:

1. Generate n random integers (use `random_array(n, seed=seed)` from `starter_code`).
2. Push all n values onto a `MinHeap` (Problem 12) and a `SortedListPQ`, in the same order.
3. Pop all n values from each (in increasing order).
4. Return `(heap_ops, sorted_list_ops)`, where `heap_ops` is the total number of "element moves" performed by the heap (count 1 per swap during sift up/down), and `sorted_list_ops` is `SortedListPQ.ops`.

Expected behavior. For increasing n (e.g. $16, 64, 256$), `heap_ops` grows roughly like $n \log n$ while `sorted_list_ops` grows roughly like n^2, and `sorted_list_ops` > `heap_ops` for $n = 256$.

Problem 15 – Dynamic Array with Doubling: Amortized Analysis

Implement `DynamicArray`, a simple dynamic array (manual resizing, no use of Python's built-in list growth) with:

- `__init__(self, initial_capacity=1)`: starts with `self.capacity = initial_capacity`, `self.size = 0`, and `self.data = [None] * self.capacity`. Also `self.copy_ops = 0` (total number of element copies performed across all resizes).
- `append(x)`: if `self.size == self.capacity`, first *resize*: create a new array of capacity `2 * self.capacity`, copy all `self.size` existing elements into it (incrementing `self.copy_ops` by `self.size` for this resize), and replace `self.data`. Then store `x` at index `self.size` and increment `self.size`.
- `__len__`: returns `self.size`.
- `__getitem__(i)`: returns `self.data[i]`.

Then implement `total_copy_ops(n)` that creates a fresh `DynamicArray()` (starting capacity 1), appends n items, and returns `self.copy_ops`.

Expected behavior. Resizes happen when size passes $1, 2, 4, 8, \ldots$ (powers of two), so `total_copy_ops(n)` equals $1 + 2 + 4 + \cdots + 2^{k-1} = 2^k - 1$ where 2^k is the smallest power of two $\geq n$ – always $< 2n$, illustrating $O(1)$ amortized `append`.

4 Algorithm Analysis Case Studies

Problem 16 – Efficient Gale–Shapley with Arrays and Queues

Week 1's Gale–Shapley implementation used Python dicts and `.index()` lookups, which can hide costly linear scans. This problem re-implements the men-proposing algorithm (Kleinberg & Tardos, Section 2.3 style) using only arrays (lists) and a queue, to make the $O(n^2)$ bound explicit and measurable.

Represent men and women as integers $0..n-1$. You are given:

- `men_pref[m]` = list of women in order of preference (best first).
- `women_pref[w]` = list of men in order of preference (best first).

Implement `build_women_rank(women_pref, n)` that returns `women_rank` where `women_rank[w][m]` is the 0-based rank of man m in woman w's list. This precomputed table lets you compare two suitors in $O(1)$ instead of calling `.index()` (which is $O(n)$).

Then implement `efficient_gale_shapley(men_pref, women_pref, n)`:

1. Build `women_rank` via `build_women_rank`.
2. Use a queue (`collections.deque`) of free men, initially containing all of $0..n-1$.
3. Maintain `next_proposal[m]` = index into `men_pref[m]` of the next woman m will propose to (starts at 0, array of size n).
4. Maintain `woman_partner[w]` = current partner of woman w, or -1 if free (array of size n).
5. While the queue is non-empty: pop a free man m, let w be his next proposal, increment `next_proposal[m]` and the total-proposals counter; if w is free, engage (m, w); else compare `women_rank[w][m]` against `women_rank[w][current partner]` and either engage (m, w) (freeing the old partner, who re-enters the queue) or re-enqueue m.

Return `(matching, total_proposals)` where `matching[m]` = woman matched to man m.

Expected behavior. The result is a valid perfect matching (each `matching[m]` distinct and covering $0..n-1$), and `total_proposals` satisfies $0 < \texttt{total\_proposals} \leq n^2$.

Problem 17 – Correctness via Induction: Sum and Power

Implement `iterative_sum(n)`, which computes $0 + 1 + \cdots + n$ using a loop that maintains the invariant "after processing i, `total == 0+1+...+i`". Return the final total (for $n \geq 0$; return 0 if $n < 0$).

Implement `fast_power(base, exp)`, which computes `base ** exp` for `exp >= 0` using repeated squaring (binary exponentiation):

$$\texttt{fast\_power(base, exp)}: \quad \begin{cases} \text{if } exp = 0: & \text{return } 1 \\ half = \texttt{fast\_power(base, exp // 2)} & \\ \text{if } exp \text{ even}: & \text{return } half \cdot half \\ \text{else}: & \text{return } half \cdot half \cdot base \end{cases}$$

This runs in $O(\log exp)$ multiplications rather than $O(exp)$.

Then implement `verify_sum_invariant(n)` that re-implements `iterative_sum` but additionally checks, *at every step* of the loop, that the invariant `total == i*(i+1)//2` holds (using `assert`), and returns the final total. If the invariant is ever violated, the assertion raises and propagates (no `try/except` inside this function).

Then implement `verify_fast_power(values)` where `values` is a list of `(base, exp)` pairs, returning `True` iff `fast_power(base, exp) == base ** exp` for every pair.

Expected behavior. `iterative_sum(100) == 5050`, `fast_power(2,10) == 1024`, `verify_sum_invariant(5 == 1275`, and `verify_fast_power([(2,0),(2,10),(3,5),(10,6),(1,100),(7,7)]) == True`.